

MEMORY ALLOCATION

```
#include <stdio.h>
#include <string.h>

#define MAX_BLOCKS 20
#define MAX_PROCESSES 20

void printAllocation(int processSize[], int n_processes, int allocation[]) {
    printf("\nProcess No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n_processes; i++) {
        printf(" %d\t\t%d\t\t", i + 1, processSize[i]);
        if (allocation[i] != -1) {
            printf("%d\n", allocation[i] + 1);
        } else {
            printf("Not Allocated\n");
        }
    }
}

void firstFit(int blockSize[], int m_blocks, int processSize[], int n_processes) {
    int allocation[MAX_PROCESSES];
    int blocks[MAX_BLOCKS];

    memcpy(blocks, blockSize, sizeof(blocks[0]) * m_blocks);
    memset(allocation, -1, sizeof(allocation));

    for (int i = 0; i < n_processes; i++) {
        for (int j = 0; j < m_blocks; j++) {
            if (blocks[j] >= processSize[i]) {
                allocation[i] = j;
                blocks[j] -= processSize[i];
                break;
            }
        }
    }

    printf("\n--- First Fit Allocation ---");
    printAllocation(processSize, n_processes, allocation);
}

void bestFit(int blockSize[], int m_blocks, int processSize[], int n_processes) {
    int allocation[MAX_PROCESSES];
    int blocks[MAX_BLOCKS];

    memcpy(blocks, blockSize, sizeof(blocks[0]) * m_blocks);
    memset(allocation, -1, sizeof(allocation));
```

```

    for (int i = 0; i < n_processes; i++) {
        int bestIdx = -1;
        for (int j = 0; j < m_blocks; j++) {
            if (blocks[j] >= processSize[i]) {
                if (bestIdx == -1 || blocks[j] < blocks[bestIdx]) {
                    bestIdx = j;
                }
            }
        }

        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            blocks[bestIdx] -= processSize[i];
        }
    }

    printf("\n--- Best Fit Allocation ---");
    printAllocation(processSize, n_processes, allocation);
}

void worstFit(int blockSize[], int m_blocks, int processSize[], int n_processes) {
    int allocation[MAX_PROCESSES];
    int blocks[MAX_BLOCKS];

    memcpy(blocks, blockSize, sizeof(blocks[0]) * m_blocks);
    memset(allocation, -1, sizeof(allocation));

    for (int i = 0; i < n_processes; i++) {
        int worstIdx = -1;
        for (int j = 0; j < m_blocks; j++) {
            if (blocks[j] >= processSize[i]) {
                if (worstIdx == -1 || blocks[j] > blocks[worstIdx]) {
                    worstIdx = j;
                }
            }
        }

        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            blocks[worstIdx] -= processSize[i];
        }
    }

    printf("\n--- Worst Fit Allocation ---");
    printAllocation(processSize, n_processes, allocation);
}

int main() {
    int m_blocks, n_processes;
    int blockSize[MAX_BLOCKS], processSize[MAX_PROCESSES];

```

```
printf("Enter the number of memory blocks: ");
scanf("%d", &m_blocks);
printf("Enter the size of each block:\n");
for (int i = 0; i < m_blocks; i++) {
    printf("Block %d: ", i + 1);
    scanf("%d", &blockSize[i]);
}

printf("\nEnter the number of processes: ");
scanf("%d", &n_processes);
printf("Enter the size of each process:\n");
for (int i = 0; i < n_processes; i++) {
    printf("Process %d: ", i + 1);
    scanf("%d", &processSize[i]);
}

firstFit(blockSize, m_blocks, processSize, n_processes);
bestFit(blockSize, m_blocks, processSize, n_processes);
worstFit(blockSize, m_blocks, processSize, n_processes);

return 0;
}
```

OUTPUT:

Enter the number of memory blocks: 5

Enter the size of each block:

Block 1: 100

Block 2: 500

Block 3: 200

Block 4: 300

Block 5: 600

Enter the number of processes: 4

Enter the size of each process:

Process 1: 212

Process 2: 417

Process 3: 112

Process 4: 426

--- First Fit Allocation ---

Process No.	Process Size	Block No.
1	212	2
2	417	5
3	112	2
4	426	Not Allocated

--- Best Fit Allocation ---

Process No.	Process Size	Block No.
1	212	4
2	417	2
3	112	3
4	426	5

--- Worst Fit Allocation ---

Process No.	Process Size	Block No.
1	212	5
2	417	2
3	112	5
4	426	Not Allocated

Process returned 0 (0x0) execution time : 56.619 s

Press any key to continue.

|

```

int main() {
    int m-blocks, n-processes;
    int blockSize [MAX-BLOCKS] (MAX-PROCESSES);
    printf("Enter the number of memory blocks: ");
    scanf("%d", &m-blocks);
    printf("Enter the size of each block: \n");
    for (int i=0; i < m-blocks; i++) {
        printf("Block %d: ", i+1);
        scanf("%d", &blockSize[i]);
    }
    printf("\nEnter the number of processes: ");
    scanf("%d", &n-processes);
    printf("Enter the size of each process: \n");
    for (int i=0; i < n-processes; i++) {
        printf("Process %d: ", i+1);
        scanf("%d", &processSize[i]);
    }
    firstFit (blockSize, m-blocks, processSize, n-processes);
    bestFit (blockSize, m-blocks, processSize, n-processes);
    worstFit (blockSize, m-blocks, processSize, n-processes);

    return 0;
}

```

OUTPUT: Enter the number of memory blocks: 5
 Enter the size of each block: 100
 500
 200
 300
 600
 Enter the number of memory blocks: 4
 Enter the size of each process: 212
 417
 112
 426

Process No.	First Fit Allocation Process Size	Block No
1	212	2
2	417	5
3	112	2
4	426	Not Allocated

Process No.	Best Fit Allocation Process Size	Block No
1	212	4
2	417	2
3	112	3
4	426	5

20/10/20

Memory Allocation

```

#include <stdio.h>
#include <string.h>
#define MAX-B 20
#define MAX-P 20

void printAllocation (int processSize[], int n-processes,
    int allocation[]) {
    printf("\nProcess No. \t Process size \t Block No. \n");
    for (int i=0; i < n-processes; i++) {
        printf(" %d \t %d \t %d", i+1, processSize[i],
            allocation[i] != -1 ? allocation[i] : -1);
        printf("\n");
    }
    else {
        printf("Not allocated \n");
    }
}

void firstFit (int blockSize[], int m-blocks, int
    processSize[], int n-processes) {
    int allocation [MAX-P];
    int blocks [MAX-B];
    memcpy (blocks, blockSize, sizeof (blocks[0]) * m-blocks);
    memset (allocation, -1, sizeof (allocation));
    for (int i=0; i < n-processes; i++) {
        for (int j=0; j < m-blocks; j++) {
            if (blocks[j] >= processSize[i]) {
                allocation[i] = j;
                blocks[j] -= processSize[i];
                break;
            }
        }
    }
    printf("\n --- First Fit Allocation ---");
    printAllocation (processSize, n-processes, allocation);
}

```

```

void bestFit (int blockSize[], int m-blocks, int
processSize[], int n-processes) {
    int allocation[MAX-P];
    int blocks [MAX-B];

    memcpy (blocks, blockSize, sizeof(blocks[0])*m-blocks);
    memset (allocation, -1, sizeof (allocation));

    for (int i=0; i < n-processes; i++) {
        int bestIdx = -1;
        for (int j=0; j < m-blocks; j++) {
            if (blocks[j] >= processSize[i]) {
                if (bestIdx == -1 || blocks[j] < blocks[bestIdx])
                    bestIdx = j;
            }
        }

        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            blocks[bestIdx] -= processSize[i];
        }

        printf ("In --- Best Fit Allocation ---");
        printAllocation (processSize, n-processes, allocation);
    }

    void worstFit (int blockSize[], int m-blocks, int
processSize[], int n-processes) {
        int allocation [MAX-P];
        int blocks [MAX-B];

        memcpy (blocks, blockSize, sizeof (blocks[0])*m-blocks);
        memset (allocation, -1, sizeof (allocation));

        for (int i=0; i < n-processes; i++) {
            int worstIdx = -1;
            for (int j=0; j < m-blocks; j++) {
                if (blocks[j] >= processSize[i]) {
                    if (worstIdx == -1 || blocks[j] > blocks[worstIdx])
                        worstIdx = j;
                }
            }

            if (worstIdx != -1) {
                allocation[i] = worstIdx;
                blocks[worstIdx] -= processSize[i];
            }

            printf ("In --- Worst Fit Allocation ---");
            printAllocation (processSize, n-processes, allocation);
        }
    }
}

```

--- Worst Fit Allocation ---

Process No.	Process Size	Block NO.
1	212	5
2	417	2
3	112	5
4	426	Not Allocated